

WSDM Cup - Multilingual Chatbot Arena

#projects

Log

21 Dec 2024: Starting with the competition!

11:20AM: I've got the compute from jarvislabs and I am thinking to start this competition today. First task will be to experiment whether the huggingface `AutoModelForClassification` matches the performance of the unsloth's normal SFT training. If I match the performance with unsloth, there are several advantages like:

- Faster training
- Lower memory usage
- Faster inference with vLLM
- Can fit more models in the dedicated runtime (ensemble approach)
- ☒ Write a baseline with HuggingFace's `AutoModelForSequenceClassification` and train a model and submit on the given data for the current competition
- ☒ Write a baseline with unsloth normal SFT training with choices as A and B and train a model and submit
- ☒ Also if possible and time remains, try the following classification implementation of unsloth ([text_classification_scripts/unsloth_classification.ipynb at main · timothelaborie/text_classification_scripts](#))

3:53PM: Currently investigating the differences between implementation of huggingface's classification and [H20.ai](#)'s classification implementation. Will log the findings here:

All experiments performed with:

- 1000 samples
- Max length: 1024
- Epoch 1
- `AutoModelForSequenceClassification` (max memory: 10 GB)

!!image.png!!(assets/image.png)

- H2O.ai inspired implementation (max memory: 13.8 GB)

```
ttr-defined]
[56/56 19:02, Epoch 0/1]

Epoch  Training Loss  Validation Loss  Accuracy  Log Loss  Runtime  Samples Per Second  Steps Per Second
0      26.092800      4.838729      0.450000  3.843380  36.675200  2.727000  1.363000

1]: TrainOutput(global_step=56, training_loss=48.142173315797535, metrics={'train_runtime': 1157.9545, 'train_samples_per_second': 0.777, 'train_steps_per_second': 0.048, 'total_flos': 0.0, 'train_loss': 48.142173315797535, 'epoch': 0.9955555555555555})
```

- Implementation details

```
[8]: from transformers.models.qwen2.modeling_qwen2 import Qwen2PreTrainedModel
from transformers.modeling_outputs import SequenceClassifierOutputWithPast
import torch.nn as nn

class Qwen2ForSequenceClassificationCustom(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = AutoModelForCausalLM.from_pretrained(ckpt, torch_dtype=torch.bfloat16)
        self.model.config.use_cache = False
        self.model.config.pad_token_id = self.model.config.eos_token_id
        self.model = prepare_model_for_kbit_training(self.model)
        self.model = get_peft_model(self.model, lora_config)

        self.classification_head = nn.Linear(self.model.config.vocab_size, 2, bias=False)

    def forward(self, input_ids, attention_mask, labels=None):
        out = self.model(input_ids, attention_mask=attention_mask)
        logits = self.classification_head(out[0][:, -1])

        if labels is not None:
            loss_fct = nn.CrossEntropyLoss()
            loss = loss_fct(logits.view(-1, 2), labels.view(-1))

        return SequenceClassifierOutputWithPast(
            loss=loss,
            logits=logits,
        )

[9]: model = Qwen2ForSequenceClassificationCustom()
model
```

- Unsloth
 - 2 token trimmed lm_head: Accuracy: 49, loaded_from_disk: 49
 - proper vocab size lm_head: Accuracy: 51, loaded_from_disk: 51

10:50PM: Apparently a custom implementation with unsloth will not work as far as I can tell as they do some patching and compilations to transformer modules as well. I implemented a custom classifier and got this error:

```
5.0.0 for 'Trainer.__init__'. Use 'processing_class' instead.  
trainer = Trainer()
```

```
[17]: trainer_stats = trainer.train()
```

```
File /root/miniconda3/envs/py3.10/lib/python3.10/site-packages/accelerate/utils/operations.py:823, in convert_outputs_to_fp32.<locals>.forward(*args, **kwargs)  
    822 def forward(*args, **kwargs):  
--> 823     return model_forward(*args, **kwargs)
```

```
File /root/miniconda3/envs/py3.10/lib/python3.10/site-packages/accelerate/utils/operations.py:811, in ConvertOutputsToFp32.__call__(self, *args, **kwargs)  
    810 def __call__(self, *args, **kwargs):  
--> 811     return convert_to_fp32(self.model_forward(*args, **kwargs))
```

```
File /root/miniconda3/envs/py3.10/lib/python3.10/site-packages/torch/amp/autocast_mode.py:44, in autocast_decorator.<locals>.decorate_autocast(*args, **kwargs)  
    41 @functools.wraps(func)  
    42 def decorate_autocast(*args, **kwargs):  
    43     with autocast_instance:  
--> 44         return func(*args, **kwargs)
```

```
TypeError: CustomClassificationModel.forward() got an unexpected keyword argument 'num_items_in_batch'
```

This is the implementation:

Unsloth: Fast downloading is enabled = ignore downloading bars which are red colored!

```
11]: from unsloth.kernels.cross_entropy_loss import fast_cross_entropy_loss
from transformers.modeling_outputs import SequenceClassifierOutput
import torch.nn as nn
class CustomClassificationModel(nn.Module):
    def __init__(self, inference=False):
        super().__init__()

        self.model, _ = FastLanguageModel.from_pretrained(
            model_name = model_name, load_in_4bit = load_in_4bit,
            max_seq_length = max_seq_length,
            dtype = dtype,
        )

        if not inference:
            self.model = FastLanguageModel.get_peft_model(
                self.model,
                r = 64,
                target_modules = [
                    "lm_head", # can easily be trained because it has only 2 tokens
                    "q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj", ],
                lora_alpha = 16,
                lora_dropout = 0, # Supports any, but = 0 is optimized
                bias = "none", # Supports any, but = "none" is optimized
                use_gradient_checkpointing = "unsloth",
                random_state = 3407,
                use_rslora = True, # We support rank stabilized LoRA
                # init_lora_weights = 'loftq',
                # loftq_config = LoftQConfig(loftq_bits = 4, loftq_iter = 1), # And LoftQ
            )

        self.classification_head = nn.Linear(self.model.config.vocab_size, 2, bias=False)

    def forward(self, input_ids, attention_mask, labels=None):
        out = self.model(input_ids, attention_mask)
        logits = self.classification_head(out[0][:, -1].float())
        print(logits.shape)
        loss = None
        if labels is not None:
            loss = nn.CrossEntropyLoss()(logits, labels)

        return SequenceClassifierOutput(loss=loss, logits=logits)
```

- Unsloth w/ complete lm_head trainable params: 706,478,080
- Unsloth w/ 2 lm_head trainable params: 161,487,872

So here are the experimentation results so far (all with base models on instruct):

- Hf with AutoModel takes 18mins for training 1000 samples with batch size 2 and grad accum 8 (effective batch size 16). Accuracy on 100 sample evaluation 51
- Unsloth's complete token head takes (151k) output takes about 8mins with 32 batch size, Accuracy 51
- Unsloth's 2 trimmed token output takes about 8mins with 32 batch size, Accuracy 49

11:23PM: Now experimenting with 2000 samples for a better feeling:

- Unsloth complete lm_head: time → 15mins , batch_size → 48, max memory: 20.47GB
 - Observations: initial loss is very high around 18 and comes less than 0.9 in around 5 steps
 - Accuracy: 54%
- Unsloth 2 token lm_head: time → 15mins, batch_size → 48, max memory: 18GB
 - Observations: initial loss is 0.9 and stable
 - Accuracy: 46%
 - This came 2% lower in 1000 training samples as well, maybe differential learning rate will work here ?

Next steps:

- ☒ Take the classification notebook and train complete lm head and compare the results
- ☒ Use instruct versions and proper templating and see the results for complete lm_head

22 Dec 2024: Doing more experiments and finalizing the stuffs

12:28PM: I was looking at the top solutions and found that most of them used base models for finetuning. Here's the relevant section from the finetuning course lecture, [Workshop 1 When and Why to Fine-Tune an LLM](#):

Instruction finetuned models are already been finetuned to speak with you in a chat like manner. You ask it a question it gives an answer. It's not like base models (which do text completion). The use case of finetuning is mostly like a very narrow use-case where mostly we don't want a chatbot or even a prompt. It's just like input and output.

Also templating will be a lot more complex to handle in cases of truncation for instruct versions and we have a lot of data for fine-tuning so using base models make sense for this type of tasks. Also the instruct versions are chat model and we need to predict only one token so

HF plain

```
[28/28 07:30, Epoch 0/1]
Epoch Training Loss Validation Loss Accuracy Log Loss Runtime Samples Per Second Steps Per Second
0 19.337400 3.331619 0.540000 3.235071 32.542000 3.073000 1.536000

Trainer.tokenizer is now deprecated. You should use Trainer.processing_class instead.
6]: TrainOutput(global_step=28, training_loss=20.801466064793722, metrics={'train_runtime': 468.34, 'tra
in_samples_per_second': 0.961, 'train_steps_per_second': 0.06, 'total_flos': 1.4664308295684096e+16,
'train_loss': 20.801466064793722, 'epoch': 0.49777777777777776})

]: %%time
```

okay final results on my dataset here with unsloth 1000 samples, 100 for evaluation:

- all token lm head: Accuracy: 51, loaded_from_disk: 51
- 2 token lm head: Accuracy: 49

25 December 2024: Building a complete workflow!

25 December 2024, 11:01: Before finalizing everything, I will be running an experiment with unsloth. I trained a simple hf baseline on A6000 and will be running unsloth on the same GPU configuration and will see if it matches the performance. I've code ready it's just matter of running it. Experiment will be on Qwen 2.5 1.5B model.

Okay here's one idea I can try. It's certain that we can't do unsloth inference on kaggle offline as the API doesn't seem to support it. So, for loading the trimmed lm head weights, I can first load the base model, then resize the lm_head to just contain two tokens and then load the PeftModel. This might work, try this approach and see if it have some performance hits.

25 December 2024, 12:04: Finetuning complete lm_head with unsloth was still very fast given there were a ton of more parameters to tune compared to `AutoModelForSequenceClassification` from huggingface. With unsloth the finetuning was atleast 2.5x faster on A6000. 1hr40min for HuggingFace and 33mins for unsloth with `lm_head` being tuned (150k output tokens). However the accuracy was not matching. 53 for hf and 48 (with some caveats as 200 weren't scored due to some memory leak) for unsloth.

Now I am testing the above theory of trimmed `lm_head` which will effectively match the huggingface implementation of classification models.

25 December 2024, 12:50: So I was able to load the trimmed head by resizing the layer of the hf base model and then loading `PeftModel` but it doesn't seem to return the output in the format i want. Basically it gives two outputs for each token whereas for classification I just want a single 2 output. I think juggling unsloth and all these quircks will make me a lot un-productive, its better to just use plain huggingface in this case as I will have access to almost all the techniques (like LoRA + , etc) and I will be familiar of what's happening.

```
[10]: peft_model = PeftModel.from_pretrained(model_hf, "lora_trimmed")

[13]: peft_model.to("cuda");

[17]: model_hf_classify.to("cuda");

[14]: tokenized = tokenizer("text tjhis is as a ", truncation=True, max_length=1024, return_tensors="pt")
tokenized = {k: v.to("cuda") for k, v in tokenized.items()}

[15]: peft_model(**tokenized).logits.shape

[15]: torch.Size([1, 7, 2])

[20]: model_hf_classify(**tokenized).logits.shape

[20]: torch.Size([1, 2])
```

Final judgement: For now I've decided to go ahead with HuggingFace implementation as I have more confidence in it and exactly know what's happening inside. With unsloth there's a lot of uncertainty. Yes, tuning models will take more time and will be more expensive but that's part of the game.

25 December 2024, 13:59: Building a huggingface baseline in a notebook, will later scale to a script:

- ☒ Check how the stuff is tokenized and what happens with tokenizer padding side left vs right

25 December 2024, 16:13: I've built and understood the baseline in depth for huggingface and will be porting everything to script and will do a full run.

For now I am reading the solutions and collecting the hyperparameters from the top teams to save some time experimenting on those.

PSI

- learning_rate: `7.0e-05`
- lora_alpha: `64`
- lora_r: `64`

- lora_dropout: 0.0
- max_length: 2048
- batch_size: 16

Mark

27 Dec 24: Noting down all configurations and hyperparameters!

Today's goal would be to note down all the configurations and hyperparams top teams used because in my experience yesterday LoRA is very very sensitive to learning rates. Also goal would be to understand how the learning rate affects the weight updates.

1st place solution LMSYS

- lora r: 64
- lora alpha: 128
- batch size: 64
- learning rate: 1e-5 or 5e-5

4th place solution LMSYS

- lora r: 64
- lora alpha: 16
- batch size: 16
- learning rate: 2e-4

5th place (PSI) LMSYS solution

- lora r: 64
- lora alpha: 64
- batch size: 16
- learning rate: 7e-5 for lora and 1e-5 for head

3rd place LMSYS solution

Raja

- lora r: 16
- lora alpha: 32
- batch size: 32
- learning rate: $8e-6$ and $1e-6$ for head

Mark

- lora r: 64
- lora alpha: 16
- batch size: 16
- learning rate: $1e-4$

9th place solution

- lora r: 32
- lora alpha: 64
- batch size: 16
- learning rate: $8e-5$

Chris solution

(FYI, if we use $LR=2e-4$ 1 epoch linear schedule with warmup, then $a=4$ is best alpha for all rank. If we use $LR=2e-5$, then $a=192$ is best alpha for all rank. If we use $LR=6e-5$, then $a=64$ is best alpha for all rank).

18th place solution LMSYS

- lora r: 64
- lora alpha: 32
- batch size: 64
- learning rate: $1e-5$

Based on the above hyperparams here's what I will try:

- r: 64, alpha: 64, learning rate: $7e-5$

- ☒ Do a single run with consolidated hyperparameters
- ☒ Try differential learning rate on classification head
- ☒ Try rslora
- ☒ Try differential learning rate for lora_a, lora_b, head, embed_tokens (as in eedi winning solution: <https://github.com/rbiswasfc/eedi-mining-misconceptions/tree/main>)
- ☐ One possible experimentation would be to finetune embed_tokens layer as well (these are the embeddings of the model, the very first layer, but seems that they are kept frozen during training for now).

4:29PM: Just hand-wrote and verified the code for lora plus implementation

[Google Colab](#)

29 Dec 24: Scaling up

5:50 PM: I was scaling up to qwen 2.5 7B but apparently the model didn't converge in the initial states of training so I killed the training. I also tried to adjust learning rates and lora alpha but with no success.

![[Image.png]](assets/Image%20(2).png)

Now instead of going to qwen I am thinking to just try out gemma as it was used in the winning solutions and recently this notebook was made public which achieved 0.684 using gemma 2 with only competition data which is wild.

![[Image.png]](assets/Image%20(3).png)

I got gemma-2 with quantization working somehow and it shows 8hrs on A100 which is a lot tbh

![[Image.png]](assets/Image%20(4).png)

I am thinking to revisit unsloth as the implementation I did seems correct and the loss was also good, but there's some gap somewhere. Rather than using huggingface trainer I am going barebone pytorch with accelerate and see how can I finetune the model. If I make unsloth work it will be a major breakthrough and experimentation speed will be a lot lot faster.

10:26PM: Unsloth passes the inputs to lm_head similar to huggingface's implementation. So it's safe to just edit the lm_head after loading the model and use it as a classification model

Relevant implementation of unsloth: <https://github.com/unslothai/unsloth/blob/87f5bffc45a8af7f23a41650b30858e097b86418/unsloth/models/llama.py#L943>

30 Dec 2024: Trying unsloth again!

9:14 am: I am trying to get back to unsloth because gemma 2 9B was taking around 8 hrs on an A100 which is a lot. Even if I can bring it down to 4 hrs then it will be a very good sign.

Calculating how many A100 hours I will require if I use all the available compute:

- Modal labs: \$1000 (A100: 2.78/h) [Hours: 360]
- HuggingFace Spaces: \$1000 (A100 80GB: 4/h) [Hours: 250]
- Jarvislabs: Rs. 68000 (A100: 105/hr) [Hours: 647]
- Lambdalabs: 150 hrs
- Total: 1407

This is what I got from tokenizer loaded from autotokenizer for gemma2 9b. add_eos_token is false

!!Image.png](assets/Image%20(5).png)

2:39 PM: Unsloth just doesn't want to work for classification, for now leaving the thought of using unsloth and just focusing on experimenting with huggingface and doing meaningful experiments

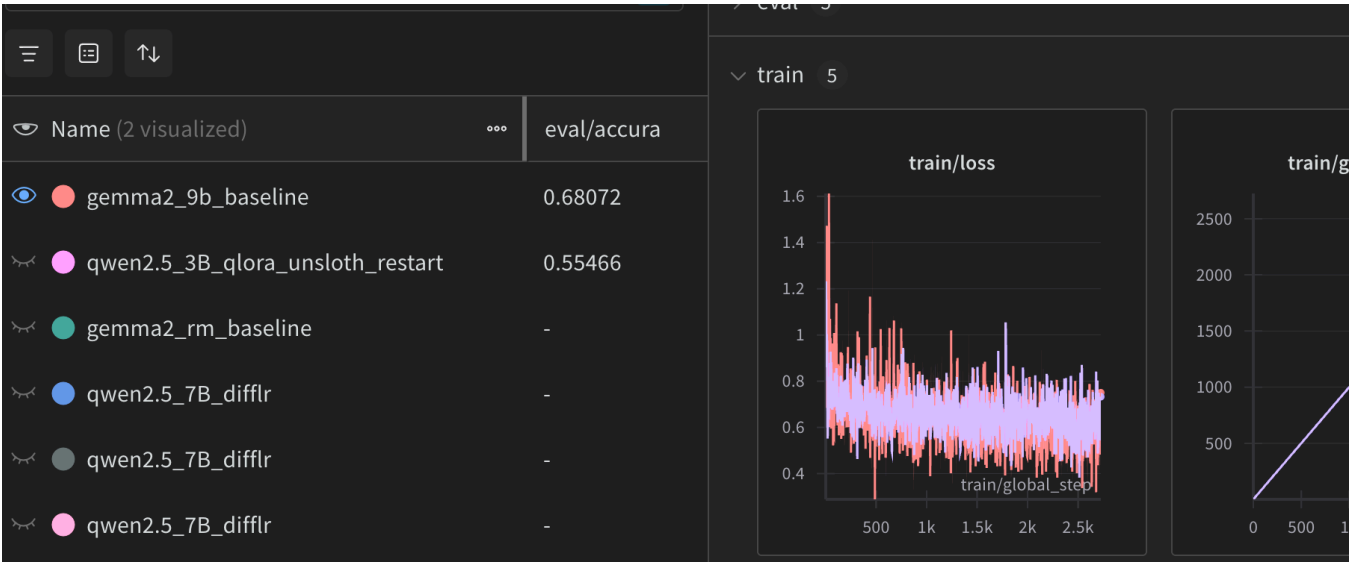
!!Image.png](assets/Image%20(6).png)

I will be doing a full run of gemma-2 9b and listing things down to get things in order:

- ☒ See the exact variant people are using (9b or 9b-it)
- ☒ See all the tokenizer related things intialized properly (eos_token: False, truncation_side, padding_side, pad_token, etc)
- ☒ Model's pad_token_id setting
- ☒ logit soft capping disable
- ☒ differential learning rate to head and body

- ☒ learning rate, rank and alpha
- ☒ make the script able to run on a multi-gpu system

12:10 am: The gemma 2 baseline scores 0.68 on CV which is wild, hope it holds on leaderboard

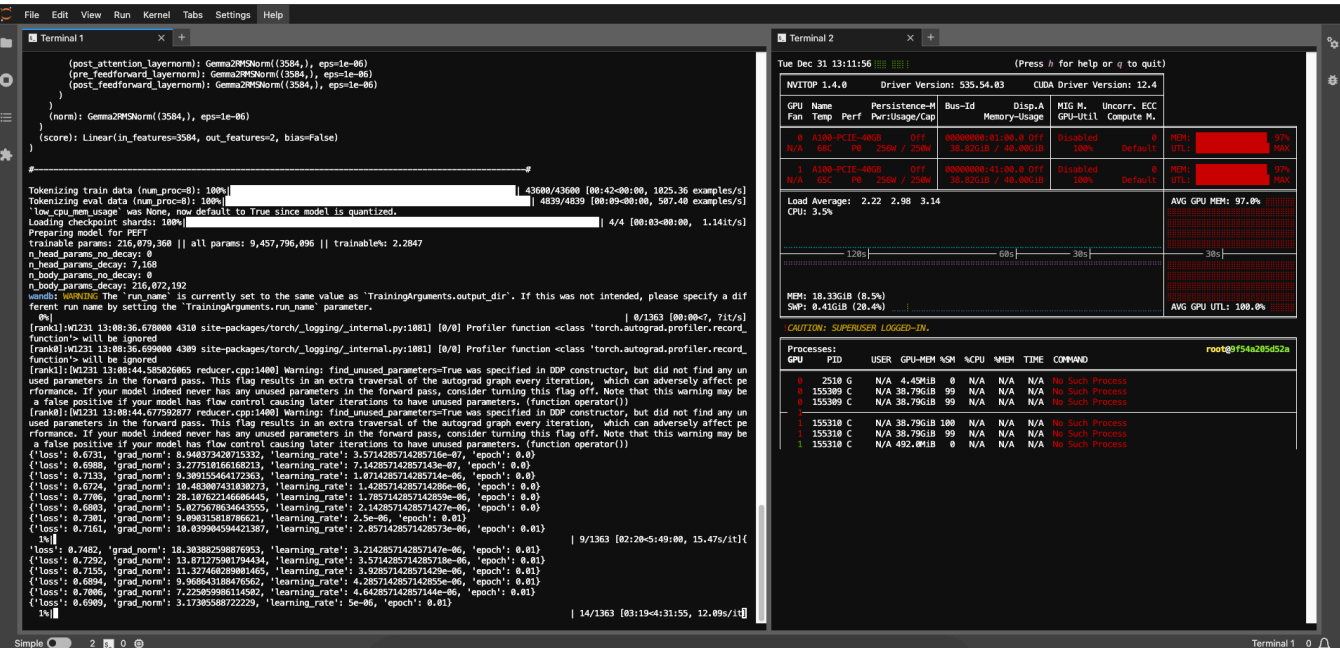


Experiment	CV	LB	Notes
qwen2.5_3B_qlora_baseline	0.634	0.641	
qwen2.5_3B_qlora_baseline	0.634	0.644	increased max length to 2048
qwen2.5_3B_qlora_baseline	0.634	0.646	increased max length to 4096
qwen2.5_3B_qlora_diff_lr_upd_hyps	0.640	0.646	4096 max length, 4 bit mode
qwen2.5_3B_qlora_diff_lr_upd_hyps	0.640	0.642	4096, 8-bit mode (suspect that model should be merged first and then do this, I directly infered on lora trained in 4bit using 8bit mode)
qwen2.5_3B_qlora_diff_lr_upd_hyps			
qwen2.5_3B_lora_plus	0.632	0.633	similar as above, infered in 8-bit

31 Dec 2024: Scaling up and reorganizing stuff.

Started training from already pretrained reward model. Once this is done, next steps will be to scale up and perform knowledge distillation with KL-divergence (soft labels) and pseudo labelling.

Training on 2 A100s



basics to do first before beginning new experiments:

- ☒ add code to save training script .py file in the checkpoint directory too
- ☒ refactor inference code to be understandable
- ☒ also make the inference code as optimized as possible (tta, dynamic batch size, dynamic padding, etc)
- ☒ make sure the new inference code matches
- ☒ try tta on the best submission and see the score

write all ideas down

- ☐ generate data with recent models for generalizing model

01 Jan 2025: Optimizing inference and collecting all the available data

7:05 pm: added tta to the best submission and submitted to leaderboard, let's see what score it gives...

Currently I am looking for different data sources and collating all the links below and will write down the ideas on how to use each data. The idea is to finish stage-1 (pretraining) atleast two models till the weekend.

Datasets

- 33k human pair preferences (was predominantly used in previous competition): https://huggingface.co/datasets/lmsys/chatbot_arena_conversations
- 3.36k samples from human judgements and 2.4k from GPT-4 judgement: https://huggingface.co/datasets/lmsys/mt_bench_human_judgments
- previous comp 55k data: <https://www.kaggle.com/competitions/lmsys-chatbot-arena/data?select=train.csv>
- lmsys 1m data (only conversations and prompts), useful for pseudo-labelling: <https://huggingface.co/datasets/lmsys/lmsys-chat-1m>
- 8.5k open model samples (multilingual) from nicholas (useful for pseudo labelling): <https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/552166>
- 48k (current competition data)

3 Jan 2025: Noting down winning solutions and starting pre-training (first-stage training)

1st place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/527629>

- Post-pretrain: Train three models llama3 70b qwen2 72b gemma2-9b on ultrafeedback dataset
- Load the weight from Post-pretrain, split the dataset into 5 folds for training, (train  4/5 kaggle train data + 33k data from lmsys, dev  1/5 kaggle train data) to train llama3 70b and qwen2 72b.
- Then infer the probability distribution of the training set.
- After obtaining the logits distribution, load the 9b model for fine-tuning and incorporate the distillation loss during the fine-tuning process. (at least three loss for training, $lr=5e-5$). [KL-divergence loss for kd]

2nd place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/527685>

- Used the extra 33k data from lmsys
- **Stage 1:** fine-tuned `google/gemma-2-9b-it`, `google/gemma-2-27b-it`, and `RLHFlow/ArmoRM-Llama3-8B-v0.1`. The validation log losses were 0.891, 0.883, and 0.899 respectively without TTA. After average ensemble, the log loss was 0.876.
- Generated pseudo labels for 240k data using the ensemble obtained from stage 1. Of this, 110k data came from lmsys-1m, 130k came from other datasets (tested numerous external datasets using a 1.5B model).
- **Stage 2:** On this dataset, fine-tuned `gemma-2-9b` and `RLHFlow/ArmoRM-Llama3-8B-v0.1`
- Based on the checkpoint obtained in stage 2, we fine-tuned using 55k (kaggle) +21k data.
- On the 20% validation set, the two models achieved 0.884 and 0.890 respectively, with the average ensemble log loss being 0.876~0.877.

3rd place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/527766>

- Start fine-tuning from a reward model instead of a chat model
- `sfairXC/FsfairX-Gemma2-RM-v0.1` instead of `google/gemma-2-9b-it`
- `RLHFlow/pair-preference-model-LLaMA3-8B` instead of `meta-llama/Meta-Llama-3.1-8B-Instruct`
- **Stage 1:** The reward models was fine-tuned on (1) filtered competition data (removed ~300 examples with high train loss), (2) 21k deduplicated **additional lmsys arena examples** and (3) a few public reward datasets (e.g. arena-hard, a subset of reward-bench and mt-bench).
- **Pseudo labelling:** collected/generated a large number of paired examples (500k+) in the competition format. Generated accompanying responses from prompt and conversations in the lmsys-1m dataset with diverse open LLMs.
- Use the soft labels from ensemble of stage 1 models for the generated dataset
- Mix the generated dataset with competition data and finetune the final models.

4th place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/529067>

- Use the official data (55k) + 33k deduplicated data, with `fold n_splits=20`, and only trained on one fold to ensure more training data
- Created pseudo-labels for 30,000 samples from the `ultrafeedback` dataset as supplementary data.
- Used the model from the first phase to create pseudo-labels for 30,000 samples from the `ultrafeedback` dataset. Then, combined this data with the first phase data (totaling over 100k) and trained a new model from scratch.

5th place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/527669>

- only use the competition data as well as the separate 33k dataset.
- However, we found that pre-training on reward data was helpful for the final models. We found this after exploring RewardBench models, and finding that some of the top models performed better than the base and instruct versions of those models when fine-tuned on competition data. Particularly the models shared by `RLHFlow` performed very well, making us curious and we saw that they trained those models on lots of UltraFeedback and similar data.
- Hence, we decided to also pre-train gemma-2-9b on public reward data, mostly UltraFeedback data. We experimented with various setups, but in the end just training on binary win/loss label was sufficient. The amount of data was also not particularly crucial, so taking all data from UltraFeedback was sufficient. Taking these pre-trained checkpoints as the starting point for our fine-tuning boosted our scores by up to 20 points on CV and public LB. Our final fine-tuned model has a local single fold score of around 0.880.
- Our final submission is an ensemble of two models, both being fine-tuned gemma-2-9b models based on our pre-trained checkpoint elaborated above. In detail, we train the same configuration two times on the full data twice for one epoch. Each run has swapped training order for response A and B for some tiny diversity. The final submission then runs both models, where one is run on the original order, and one on the swapped order for TTA. This blend has a local single fold score of around 0.873.

9th place

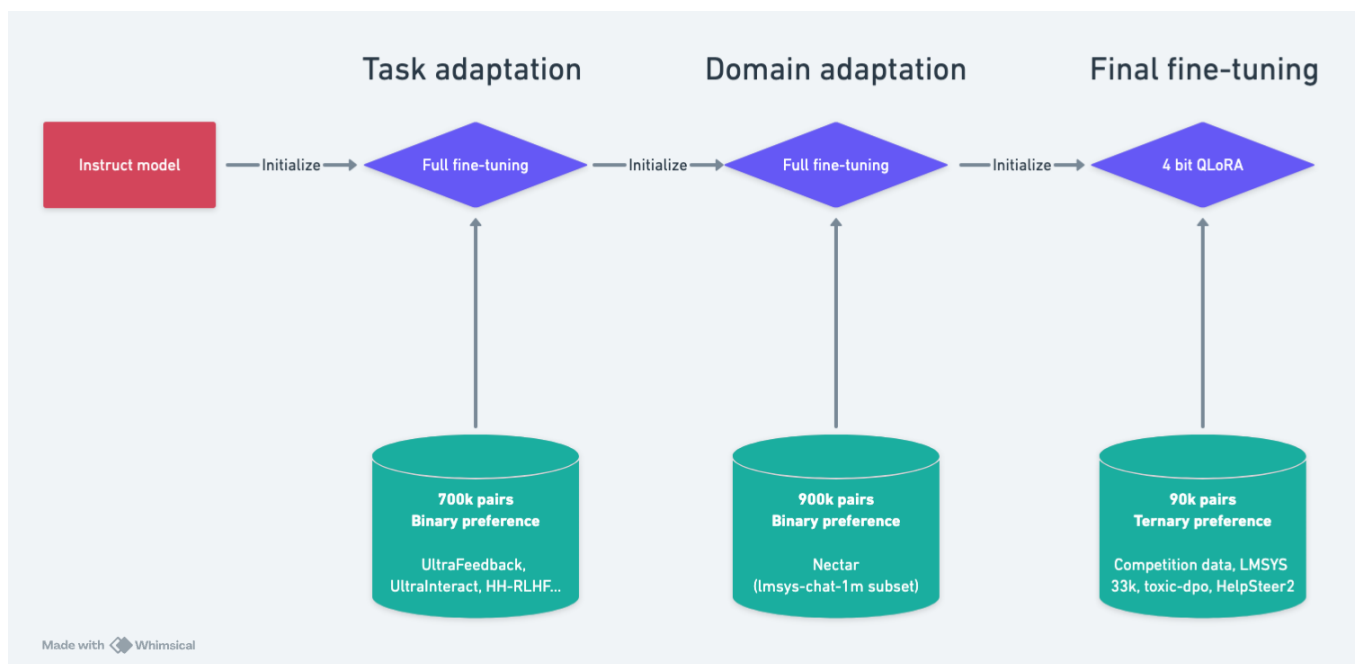
<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/527704>

- train the gemma2-9b-it using the lmsys-chat-1m pseudo label data (soft labels)

- ensemble with llama3 model
1. Train single gemma2 model and perform response swap TTA → LB : 0.890
 2. Instead of TTA, I train another gemma2 model with response swap data and ensemble it. → LB : 0.887
 3. Train both model in the same way as 2. by adding chat-1m pseudo label → LB : 0.881 (**private LB: 0.9859**)
 4. ensemble with single llama3 model without TTA (LB: 0.899 with TTA) → LB : 0.881 (**private LB: 0.9828**)
 5. For both models, I did not perform TTA (due to time constraints) and performed inference, but the model trained on swapped data also swapped data during inference.

13th place

<https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/540876>



gemma 27b pretraining is taking 21 hours even on the fastest H100 available 🤔

```

n_head_params_decay: 9,216
n_body_params_no_decay: 0
n_body_params_decay: 456,720,384
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not intend
ed, please specify a different run name by setting the `TrainingArguments.run_name` parameter.
{'loss': 0.859, 'learning_rate': 7.874015748031497e-08, 'epoch': 0.0}
{'loss': 0.5863, 'learning_rate': 1.5748031496062994e-07, 'epoch': 0.0}
{'loss': 0.691, 'learning_rate': 2.362204724409449e-07, 'epoch': 0.0}
{'loss': 0.7005, 'learning_rate': 3.149606299212599e-07, 'epoch': 0.0}
{'loss': 0.7944, 'learning_rate': 3.937007874015748e-07, 'epoch': 0.0}
{'loss': 0.9133, 'learning_rate': 4.724409448818898e-07, 'epoch': 0.0}
{'loss': 0.9057, 'learning_rate': 5.511811023622048e-07, 'epoch': 0.0}
{'loss': 0.6777, 'learning_rate': 6.299212598425198e-07, 'epoch': 0.0}
{'loss': 0.6631, 'learning_rate': 7.086614173228346e-07, 'epoch': 0.0}
{'loss': 0.6823, 'learning_rate': 7.874015748031496e-07, 'epoch': 0.0}
0%|
| 10/6327 [02:20<23:11:04, 13.21s/it]

```

```

Gemma2ForSequenceClassification
(model): Gemma2Model(
  (embed_tokens): Embedding(256000, 4608, padding_idx=0)
  (layers): ModuleList(
    (0-45): 46 x Gemma2DecoderLayer(
      (self_attn): Gemma2Attention(
        (q_proj): Linear4bit(in_features=4608, out_features=4096, bias=False)
        (k_proj): Linear4bit(in_features=4608, out_features=2048, bias=False)
        (v_proj): Linear4bit(in_features=4608, out_features=2048, bias=False)
        (o_proj): Linear4bit(in_features=4096, out_features=4608, bias=False)
        (rotary_emb): Gemma2RotaryEmbedding()
      )
      (mlp): Gemma2MLP(
        (gate_proj): Linear4bit(in_features=4608, out_features=36864, bias=False)
        (up_proj): Linear4bit(in_features=4608, out_features=36864, bias=False)
        (down_proj): Linear4bit(in_features=36864, out_features=4608, bias=False)
        (act_fn): PytorchGELUTanh()
      )
      (input_layer_norm): Gemma2RMSNorm((4608,), eps=1e-06)
      (post_attention_layer_norm): Gemma2RMSNorm((4608,), eps=1e-06)
      (pre_feedforward_layer_norm): Gemma2RMSNorm((4608,), eps=1e-06)
      (post_feedforward_layer_norm): Gemma2RMSNorm((4608,), eps=1e-06)
    )
  )
  (norm): Gemma2RMSNorm((4608,), eps=1e-06)
)
(score): Linear(in_features=4608, out_features=2, bias=False)

```

```

Preparing model for PEFT
trainable params: 456,720,600 || all params: 27,683,867,136 || trainable: 1.6498
n_head_params_decay: 0
n_head_params_no_decay: 0
n_body_params_decay: 456,720,384
wandb: WARNING The `run_name` is currently set to the same value as `TrainingArguments.output_dir`. If this was not intend
ed, please specify a different run name by setting the `TrainingArguments.run_name` parameter.
{'loss': 0.859, 'learning_rate': 7.874015748031497e-08, 'epoch': 0.0}
{'loss': 0.5863, 'learning_rate': 1.5748031496062994e-07, 'epoch': 0.0}
{'loss': 0.691, 'learning_rate': 2.362204724409449e-07, 'epoch': 0.0}
{'loss': 0.7005, 'learning_rate': 3.149606299212599e-07, 'epoch': 0.0}
{'loss': 0.7944, 'learning_rate': 3.937007874015748e-07, 'epoch': 0.0}
{'loss': 0.9133, 'learning_rate': 4.724409448818898e-07, 'epoch': 0.0}
{'loss': 0.9057, 'learning_rate': 5.511811023622048e-07, 'epoch': 0.0}
{'loss': 0.6777, 'learning_rate': 6.299212598425198e-07, 'epoch': 0.0}
{'loss': 0.6631, 'learning_rate': 7.086614173228346e-07, 'epoch': 0.0}
{'loss': 0.6823, 'learning_rate': 7.874015748031496e-07, 'epoch': 0.0}
{'loss': 0.6781, 'learning_rate': 8.6614173228346e-07, 'epoch': 0.0}
{'loss': 1.1399, 'learning_rate': 9.448818897637796e-07, 'epoch': 0.0}
{'loss': 0.8762, 'learning_rate': 1.0236220472440946e-06, 'epoch': 0.0}
{'loss': 0.9799, 'learning_rate': 1.1023622047244096e-06, 'epoch': 0.0}
0%|
| 14/6327 [03:04<19:58:49, 11.39s/it]

```

Sat Jan 04 09:10:09 2025 (Press h for help or q to quit)

NVITOP 1.4.0				Driver Version: 550.127.05				CUDA Driver Version: 12.4			
GPU	Name	Persistence-M	Bus-Id	Disp-A	Memory-Usage	MIG M.	Uncorr. ECC	GPU-Util	Compute M.		
Fan	Temp	Perf	Pwr:Usage/Cap								
0	H100	80C	HM3	On	40000000/85:88.0 Off	Disabled	0	100%	Default	0	84.3% MAX
N/A	54C	PR	691W / 700W		67.15GiB / 79.63GiB						

Load Average: 1.03 1.18 1.27
CPU: 3.8%

MEM: 6.45GiB (3.8%)
SWP: 0.00GiB (0.0%)

GPU MEM: 84.3%
GPU UTIL: 75.0%

Processes:	GPU	PID	USER	GPU-MEM	%SH	%CPU	%MEM	TIME	COMMAND
0	7327	C	ubuntu	67.14GiB	99	100.4	2.3	4:58	python src/train_gemma2.py configs/gemma2_27b_stag..

```

ubuntu@192-222-54-183: ~/WSDM-Cup-Multilingual-Chatbot-Arena-Kaggle/artifacts/gemma2_27b_rm_stage_1_baseline (ssh)
artifacts data infer_multi_gpu.py pyproject.toml run.sh src
configs infer.py notebooks requirements.txt setup.sh
ubuntu@192-222-54-183: ~/WSDM-Cup-Multilingual-Chatbot-Arena-Kaggle$ cd artifacts/
gemma2_27b_rm_stage_1_baseline
ubuntu@192-222-54-183: ~/WSDM-Cup-Multilingual-Chatbot-Arena-Kaggle/artifacts$ cd gemma2_27b_rm_stage_1_basele
e/
ubuntu@192-222-54-183: ~/WSDM-Cup-Multilingual-Chatbot-Arena-Kaggle/artifacts/gemma2_27b_rm_stage_1_basele$ l
s
README.md experiment_config.yaml tokenizer.json training_args.bin
adapter_config.json oof_preds.parquet tokenizers_config.json
adapter_model.safetensors special_tokens_map.json train.py
ubuntu@192-222-54-183: ~/WSDM-Cup-Multilingual-Chatbot-Arena-Kaggle/artifacts/gemma2_27b_rm_stage_1_basele$

```

6:12PM : finally got modal labs training working, training a gemma 27B stage 1 model on 2xH100 now:

gladiator07 Starter / main

Logs

Secrets

Storage

Sandboxes

Live Usage: \$10.48

Slack Community

Docs

Apps

Quickstart guide

Search (1K)

Deployed Apps 0 items

Ephemeral Apps 1 item

Stopped Apps 22 items

Sort: Most recent

modal_run_train_job.py

started 18 minutes ago

run_training

2 x H100 GPU

1 container, 1 input

```
Filtering by function: run_training X
Jan 04 18:12:57.617 { 'loss': 0.509, 'learning_rate': 1.9975e-05, 'epoch': 0.02}
Jan 04 18:12:57.618 2%|          | 62/3164 [09:06<7:42:30, 8.95s/it]
Jan 04 18:13:04.454 2%|          | 63/3164 [09:13<7:09:44, 8.31s/it]
Jan 04 18:13:04.456 {'loss': 0.6318, 'learning_rate': 1.96875e-05, 'epoch': 0.02}
Jan 04 18:13:04.459 2%|          | 63/3164 [09:13<7:09:44, 8.31s/it]
Jan 04 18:13:04.461 2%|          | 64/3164 [09:20<6:40:27, 7.75s/it]
Jan 04 18:13:10.889 {'loss': 0.5828, 'learning_rate': 2e-05, 'epoch': 0.02}
Jan 04 18:13:16.763 2%|          | 65/3164 [09:25<6:11:15, 7.19s/it]
Jan 04 18:13:16.765 {'loss': 0.803, 'learning_rate': 1.99999486493051e-05, 'epoch': 0.02}
Jan 04 18:13:16.768 2%|          | 65/3164 [09:25<6:11:15, 7.19s/it]
Jan 04 18:13:16.770 2%|          | 66/3164 [09:31<5:44:13, 6.67s/it]
Jan 04 18:13:22.213 {'loss': 0.4358, 'learning_rate': 1.9999979459727323e-05, 'epoch': 0.02}
Jan 04 18:13:22.218 2%|          | 66/3164 [09:31<5:44:13, 6.67s/it]
Jan 04 18:13:22.220 2%|          | 67/3164 [09:36<5:16:35, 6.13s/it]
Jan 04 18:13:27.102 {'loss': 0.6896, 'learning_rate': 1.9999953784406248e-05, 'epoch': 0.02}
Jan 04 18:13:27.104 2%|          | 67/3164 [09:36<5:16:35, 6.13s/it]
Jan 04 18:13:27.107 2%|          | 68/3164 [09:40<4:48:35, 5.59s/it]
Jan 04 18:13:31.435 {'loss': 0.485, 'learning_rate': 1.9999917838993664e-05, 'epoch': 0.02}
Jan 04 18:13:31.437 2%|          | 68/3164 [09:40<4:48:35, 5.59s/it]
Jan 04 18:13:31.440 2%|          | 69/3164 [09:44<4:24:27, 5.13s/it]
Jan 04 18:13:35.474 {'loss': 0.5933, 'learning_rate': 1.9999871623526484e-05, 'epoch': 0.02}
Jan 04 18:13:35.476 2%|          | 69/3164 [09:44<4:24:27, 5.13s/it]
Jan 04 18:13:35.479 2%|          | 69/3164 [09:44<4:24:27, 5.13s/it]
Jan 04 18:13:35.481 2%|          | 69/3164 [09:44<4:24:27, 5.13s/it]
----- listening for logs -----
```

gemma stage 1: 0.701

llama3 stage 1: 0.687

Writing timeout submissions info:

Experiment	CV	LB
gemma 2 9b baseline, 4096	0.68072	0.679
gemma 2 9b reward model baseline, 4096	0.6927	0.682
same as above with tta, 4096	-	timeout
same as above with tta, 2048	-	timeout
same as above with tta, 1800	-	timeout
gemma 2 rm baseline, 1024, with tta	0.6927	0.689
gemma 2 rm baseline, 1536, with tta	0.6927	timeout

gemma 2 rm baseline, with only tta 4096 (no other, one model)	0.6927	0.687
gemma 2 truncate middle, 1024, tta	0.696	0.682

Maybe I should take inspiration from this notebook and do tta and inference similar to it, it might be more efficient since it sorts the complete tta and non-tta data and batches it together, might be more efficient (will also save 5-10 minutes of multiple times loading model)

[[WSDM Cup] LB:0.684 Only Gemma-2 9b 4-bit](<https://www.kaggle.com/code/takaito/wsdm-cup-lb-0-684-only-gemma-2-9b-4-bit/notebook>)

checkout this too: <https://claude.ai/chat/45eab3d5-7ea9-4535-8d3a-5c1baabb12f3>

Models to run on huggingface serverless inference:

- deepseek-ai/DeepSeek-V3
- Qwen/Qwen2.5-72B-Instruct
- Nexusflow/Athene-V2-Chat
- mistralai/Mixtral-8x7B-Instruct-v0.1
- meta-llama/Llama-3.3-70B-Instruct
- Qwen/Qwen2.5-Coder-32B-Instruct
- mistralai/Mistral-Nemo-Instruct-2407
- meta-llama/Llama-3.1-70B-Instruct
- EleutherAI/gpt-neox-20b
- 01-ai/Yi-1.5-34B-Chat
- Qwen/QwQ-32B-Preview
- HuggingFaceH4/starchat2-15b-v0.1
- google/gemma-2-27b-it
- meta-llama/Meta-Llama-3-8B-Instruct

Models to run on VLLM:

- Qwen/Qwen2-7B-Instruct
- microsoft/Phi-3-small-8k-instruct
- 01-ai/Yi-1.5-9B-Chat
- Qwen/Qwen1.5-14B-Chat-AWQ

- meta-llama/Llama-3.2-1B
- meta-llama/Llama-3.2-3B-Instruct
- meta-llama/Llama-3.1-8B-Instruct
- Qwen/Qwen2-1.5B-Instruct
- alpaca-13b
- mistralai/Mistral-7B-Instruct-v0.2
- mistralai/Mistral-7B-Instruct-v0.3
- Qwen/Qwen2-7B-Instruct
- microsoft/Phi-3-mini-4k-instruct
- HuggingFaceTB/SmolLM-1.7B-Instruct
- google/gemma-2-2b-it
- google/gemma-2-9b-it
- Qwen/Qwen2.5-3B-Instruct
- Qwen/Qwen2.5-7B-Instruct
- Qwen/Qwen2.5-14B-Instruct-AWQ
- google/gemma-2-27b-it

Run command:

```
python scripts/vllm_generate.py --ignore_ids_path "../vllm_inference_ids.pkl"
--save_name "lmsys_1m_5k_debug" --out_dir "./artifacts" --debug
```

- **gemma stage 1 merged 9b** (accuracy: 0.705) (0.706 with tta) [$0.6 * \text{logits} + 0.4 * \text{logits_tta} = 0.708$]
- **llama3_8b_stage1_merged** (accuracy: 0.695) (0.699 with tta) (llama best result with 0.5 weight)
- ensemble with weight of $0.8 * \text{gemma} + 0.2 * \text{llama} = 0.7092$
- ensemble of tta logits: $0.7878 * \text{gemma_tta_ensemble} + 0.2122 * \text{llama_tta_ensemble} = 0.71089$

```
100%|██████████| 605/605 [07:51<00:00, 2.38s/it]
{'eval_loss': 0.562389433839417, 'eval_model_preparation_time': 0.0048, 'eval_accuracy': 0.6945649927671006, 'eval_runtime': 472.5229, 'eval_samples_per_second': 10.241, 'eval_steps_per_second': 1.28}
100%|██████████| 605/605 [07:51<00:00, 1.28it/s]
App completed. View run at https://modal.com/apps/gladicator07/main/ap-EA1zbawudnLU9lwnNFX1rtU
```

```
gt_labels = df["labels"].tolist()
logits_1 = np.array(df["logits"].tolist())
accuracy_score(gt_labels, logits_1.argmax(-1))
```

25] ✓ 0.0s

0.7042777433353998

```
df1 = pd.read_parquet("../data/eval_data_llama3_stage1_logits.parquet")
df1 = df1.sort_values("id").reset_index(drop=True)
df1["labels"] = df1["winner"].map({"model_a": 0, "model_b": 1})
gt_labels = df1["labels"].tolist()
logits_2 = np.array(df1["logits"].tolist())
accuracy_score(gt_labels, logits_2.argmax(-1))
```

27] ✓ 0.0s

0.6951849555693325

```
ensemble_logits = 0.8 * logits_1 + 0.2 * logits_2
accuracy_score(gt_labels, ensemble_logits.argmax(-1))
```

42] ✓ 0.0s

0.7092374457532548

Plan:

- Filter only high confidence predictions
- Convert hard labels (stage 1 data) into soft labels using label smoothing
- Retrain the model with stage 1 + pseudo labelled data with dense cross entropy loss with soft labels.
- Check if any other data can be added (ORPO-DPO, etc)

Can also try bradly terry loss ?

FYI

here's how the logits are prepared:

gemma 2 stage 1 pseudo labels

$\text{gemma_logits} = 0.6 * \text{logits} + 0.4 * \text{logits_tta}$

llama3 stage 1 pseudo labels

$\text{llama_logits} = 0.5 * \text{logits} + 0.5 * \text{logits}$

final soft labels

$\text{logits} = 0.7878 * \text{gemma_logits} + 0.2122 * \text{llama_logits}$

$\text{soft_labels} = \text{softmax}(\text{logits})$

11 jan 2025: running the big experiment

DONT FORGET TO SCALE THE LEARNING RATE FOR MULTI-GPU SETUP.

GPUs go brrr.....

Sun Jan 12 11:03:36 2025

(Press **h** for help or **q** to quit)

NVITOP 1.4.0			Driver Version: 535.54.03			CUDA Driver Version: 12.4		
GPU	Name	Persistence-M	Bus-Id	Disp.A	MIG M.	Uncorr. ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.		
0	A100-PCIE-40GB	Off	00000000:A1:00.0	Off	Disabled	0	MEM:	96%
N/A	67C	P0	250W / 250W	38.55GiB / 40.00GiB	100%	Default	UTL:	MAX
1	A100-PCIE-40GB	Off	00000000:C1:00.0	Off	Disabled	0	MEM:	96%
N/A	69C	P0	242W / 250W	38.56GiB / 40.00GiB	100%	Default	UTL:	MAX
2	A100-PCIE-40GB	Off	00000000:C4:00.0	Off	Disabled	0	MEM:	96%
N/A	70C	P0	278W / 250W	38.56GiB / 40.00GiB	100%	Default	UTL:	MAX
3	A100-PCIE-40GB	Off	00000000:E1:00.0	Off	Disabled	0	MEM:	91.4%
N/A	66C	P0	228W / 250W	36.54GiB / 40.00GiB	100%	Default	UTL:	MAX
Load Average: 7.48 7.51 7.50							AVG GPU MEM: 95.1%	
CPU: 12.1%								

- select the data with only high confidence
- finetune gemma 27B and do knowledge distillation (this will be game changer if successful)

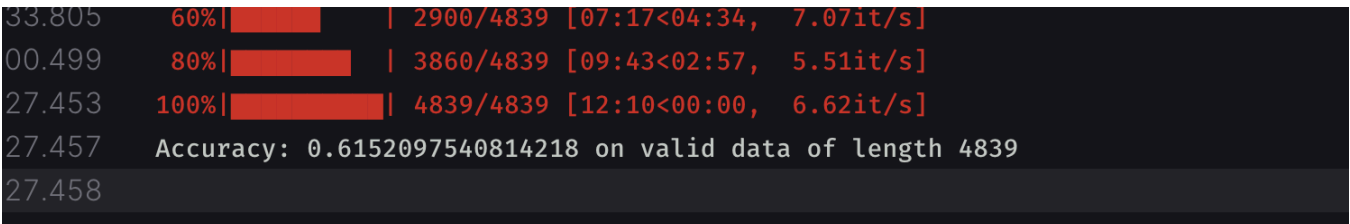
One more reason I think that 27B will be a game changer if I am able to finetune it successfully, people on reddit report that it is shockingly good for low resource multilingual language

Evaluation of Gemma 2 9B and 27B on LMSYS Chatbot Arena

before 27B I should really try finetuning in the reward setting

https://github.com/RLHFlow/RLHF-Reward-Modeling/blob/main/bradley-terry-rm/llama3_8B_rm.py

apparently llama 3 reward model scores badly on eval data without finetuning



Submissions on 14 Jan 24

Doing a bunch of submissions today

Link	Comment	CV	LB
reward model inference v1	1024 max len, 8bit		failed (i guess I set batch size for (2048, 1024) as 4 (need 2 in my local testing)
reward model inference v2	1024 max len, 4bit, just to compare results with 8bit		
reward model inference v3	1536 max len, 8bit		

best sub 8bit v1	best sub till now, gemma 2 stage 2, 4096 max len, 8bit		exception (used 1 batch size for 4096 , then why ??)
best sub 8bit v2	best sub tta, stage 2, 2048 max len, tta=true		

15 Jan 25: will not give up!

most of the experiments failed in the last week with no improvements, again trying...

Next steps to try:

- ☐ Retrain stage 1 gemma 2 model with updated data (plus find good quality datasets for preference). Train with a max length of 2048 and dropout of 0.05 for gemma
- ☐ Do similar to llama 3 model as above
- ☐ Perform pseudo labelling again with the updated stage 1 models
 - ☐ Try taking only high confidence predictions for retraining the models
- ☐ try gemma 2 27B again with lower learning rate (got 0.689 score) with the following config and updated stage 1 data
- ☐ Try vllm inference for kaggle submission, experiencing lot of timeouts. vllm does support classification (see: https://docs.vllm.ai/en/latest/models/supported_models.html#classification-task-classify)

```
debug: False
use_wandb: False
upload_to_hf_hub: False
sample_data: False
seed: -1
```

```
output_dir: ./outputs
data_dir: ./
```

```
wandb_project_name: WSDM-Cup-Multilingual-Chatbot-Arena
experiment_name: gemma2_27b_rm_stage_1_v2
```

```
notes: retry gemma 2 27b reward model stage 1 training with updated
hyperparameters
train_file_name: train_combined_stage_1_v2.parquet

model:
  model_name: Skywork/Skywork-Reward-Gemma-2-27B-v0.2
  attn_implementation: flash_attention_2
  dtype: bfloat16
  quantize_to_4bit: true
  disable_attn_logit_softcapping: False

tokenizer:
  max_length: 1536
  truncation: True
  padding_side: left
  truncation_side: left
  pad_to_multiple_of: 16

lora:
  r: 64
  alpha: 64
  target_modules: all-linear
  dropout: 0.05
  bias: none
  modules_to_save: ['score']

optimizer:
  name: AdamW8bit
  head_lr: 2.0e-5
  lr: 5.0e-5

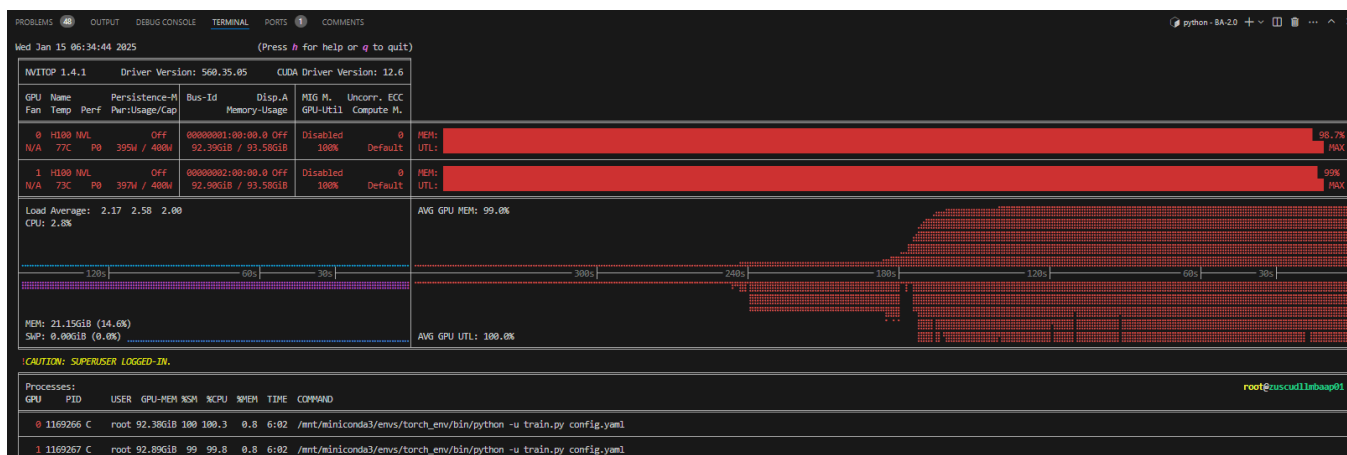
trainer_args:
  eval_strategy: epoch
  save_strategy: "no"
  logging_strategy: steps
  logging_first_step: True
  logging_steps: 1
  eval_on_start: false
  per_device_train_batch_size: 16
  per_device_eval_batch_size: 16
  gradient_accumulation_steps: 3
  learning_rate: ${optimizer.lr}
```

```

num_train_epochs: 1
lr_scheduler_type: "cosine"
warmup_ratio: 0.02
bf16: true
tf32: true
bf16_full_eval: true
group_by_length: true
gradient_checkpointing: true
gradient_checkpointing_kwargs:
  use_reentrant: false
torch_compile: true
use_liger_kernel: true
report_to: "none"
weight_decay: 0.01
max_grad_norm: null
ddp_find_unused_parameters: false

```

damn 27B is cooking



use this score head

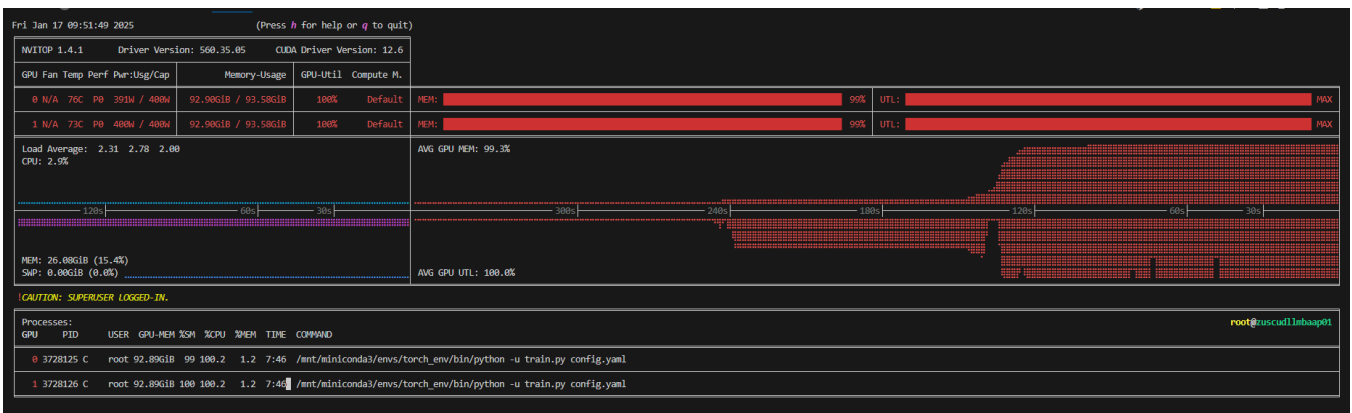
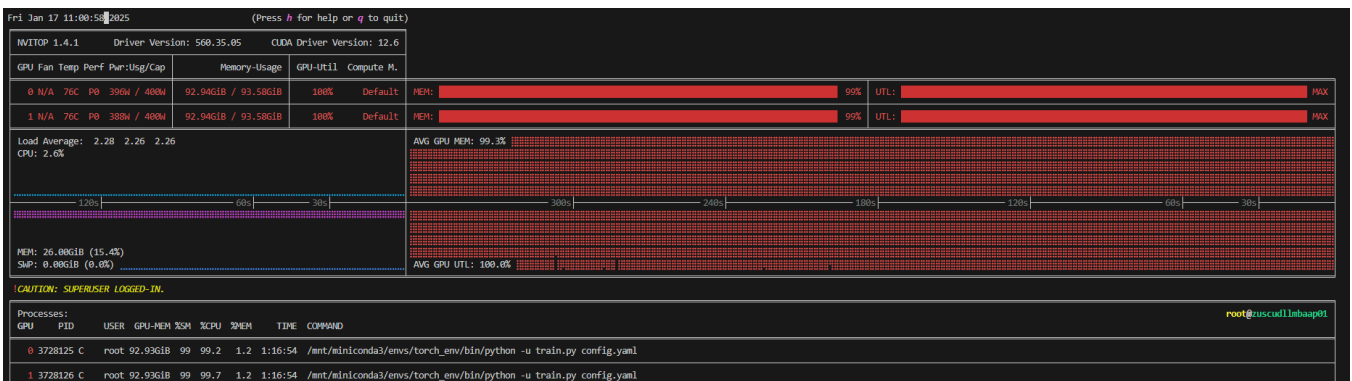
```

model.score = torch.nn.Sequential(
    torch.nn.Linear(hdim, hdim // 2),
    torch.nn.GELU(),
    torch.nn.Linear(hdim // 2, 3)
)

```

Following parameters needed to be changed before a big run for stage 1 model:

- ☒ add this data to the mix: <https://huggingface.co/datasets/C4AI-Community/multilingual-reward-bench>
- ☒ check all the dataset is properly formatted and is correct
- ☒ change max length to 2048
- ☒ change batch size and learning rate accordingly (with how many gpus you choose)
- ☒ change the model score head as above
- ☒ test it first on small data for some steps
- ☒ run the full blown experiment



- ☒ 27B model (server H100)
- ☒ 9B (parameters batch size)
- ☒ 9B (mrewardbench remove)

Jan 21: Scaling up 🔥🔥🔥

In the final 2 weeks of the competition and I really need to scale up. Here are the todos:

- ☐ Collect all the experiments you did and their CV scores at one place for better analysis

- ☒ Write down the todos for training diverse set of models and step by step approach

Finalize pseudo-labelling dataset

Goal: Finalize dataset used to train the final models with pseudo labels

- ☐ Check the data again
- ☐ Merge the samples from nicholas (large models) into your pseudo labelled samples
- ☐ Decide whether if any more data is required for final stage or not

Training strong stage 1 models and generating pseudo labels

- Stage 0: pretrain on skywork and orpo-dpo dataset
- Stage 1: take stage 0 lora, merge with base model and init the weights from this merged model and train on current comp + previous comp + lmsys 22k dataset (lmsys specific dataset)
- Take weights from stage 1 and then generate pseudo labels on extra lmsys dataset taken from lmsys-1m (with tta)
- prepare the final soft labels on pseudo labels with the ensemble of stage 1 models (with optimal weights)

Model	Stage 0	Stage 1	Generate pseudo labels (with tta)
gemma-2-9b-it	✓	✓	
gemma-2-27b-it	✓		
Deepseek R llama 3 distilled	✓		
Deepseek R qwen distilled			

After generating pseudo labels

- ☐ Ensemble all the stage 1 models in best possible way (find optimal weights)
- ☐ If the score is decent enough, then just use soft-cross entropy loss
- ☐ If the score is significantly high, then do knowledge distillation using kl-divergence

Final set of experiments

- ☐ Submit with either tta of the same model or some another strong model (or the same model trained with swap responses)
- ☐ Try distilling all the knowledge from stage 1 model's ensemble in modern bert and see if it can be added in ensemble.

Maybe hard labels is the issue ? I might have to use 3-4 losses to mitigate this (even for the proper labeled data that I have, I should create pseudo labels from ensemble models and then incorporate distillation losses)

```
loss_fun = nn.CrossEntropyLoss()
divergence_loss_fn = nn.KLDivLoss(reduction='batchmean')
cos_loss_fn = nn.CosineEmbeddingLoss()
outputs = model(batch['input_ids'], use_cache=False) # predict gemma2
logits = outputs.logits
grads = batch['grads']
grads1 = batch['grads'][:, :3] # qwen2
grads2 = batch['grads'][:, 3:] # llama3
labels = batch['labels']
loss_ce = loss_fun(logits, labels)
loss_grad1 = divergence_loss_fn(
    F.log_softmax(logits / T, dim=1),
    F.softmax(grads1 / T, dim=1)
)
cos_loss1 = cos_loss_fn(F.softmax(grads1 / T, dim=1), F.softmax(logits / T, dim=1),
    torch.ones(logits.size()[0]).to(logits.device))

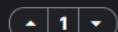
loss_grad2 = divergence_loss_fn(
    F.log_softmax(logits / T, dim=1),
    F.softmax(grads2 / T, dim=1)
)
cos_loss2 = cos_loss_fn(F.softmax(grads2 / T, dim=1), F.softmax(logits / T, dim=1),
    torch.ones(logits.size()[0]).to(logits.device))

loss = (loss_ce + loss_grad1 + cos_loss1 + loss_grad2 + cos_loss2) / 5.
```



zhuoqun Posted a month ago · 14th in this Competition

@shaomutan Hi Shaomu, I suspect that hard labels have a lot of noise and mislabeling. Maybe a baseline model is needed to generate some soft labels for training, or hard labels are trained by the operation of label smoothing.



Source: <https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/>

551267#3074985

variations of labels:

- ☒ already labelled data with label smoothing + unlabeled soft labels from stage 1 ensemble
- ☐ already labelled data soft labels from ensemble + unlabeled soft labels from stage 1 ensemble
- ☐ already labelled data with label smoothing and kl divergence loss
- ☐ already labelled data with soft labels from ensemble and kl divergence loss

GPU go brrrr....

```

Sun Jan 26 08:43:04 2025                               (Press h for help or q to quit)

+-----+
| NVITOP 1.4.1           Driver Version: 535.54.03      CUDA Driver Version: 12.4 |
+-----+
| GPU Name                Persistence-M   Bus-Id        Disp.A    MIG M.     Uncorr. ECC |
| Fan  Temp  Perf          Pwr:Usage/Cap       Memory-Usage  GPU-Util    Compute M. |
+-----+
|  0 A100-PCIE-40GB       Off            00000000:01:00:0 Off   Disabled    0         MEM: ██████████ 96% |
| N/A  67C   P0    214W / 250W             38.55GiB / 40.00GiB  100%      Default UTL: ██████████ MAX |
+-----+
|  1 A100-PCIE-40GB       Off            00000000:41:00:0 Off   Disabled    0         MEM: ██████████ 96% |
| N/A  64C   P0    252W / 250W             38.55GiB / 40.00GiB  100%      Default UTL: ██████████ MAX |
+-----+
|  2 A100-PCIE-40GB       Off            00000000:61:00:0 Off   Disabled    0         MEM: ██████████ 92.1% |
| N/A  65C   P0    290W / 250W             36.84GiB / 40.00GiB  100%      Default UTL: ██████████ MAX |
+-----+
|  3 A100-PCIE-40GB       Off            00000000:81:00:0 Off   Disabled    0         MEM: ██████████ 96% |
| N/A  65C   P0    252W / 250W             38.22GiB / 40.00GiB  100%      Default UTL: ██████████ MAX |
+-----+
| Load Average: 4.30 5.38 3.92                                     AVG GPU MEM: 95.1% ██████████ |
| CPU: 6.7%                                                         ██████████ |
|                                                                     ██████████ |
| ██████████ 120s | ██████████ 60s | ██████████ 30s |                 ██████████ 60s | ██████████ 30s |
| ██████████ | ██████████ | ██████████ | ██████████ | ██████████ | ██████████ |
| MEM: 33.43GiB (14.1%)                                             ██████████ |
| SWP: 0.42GiB (21.2%)                                             ██████████ |
| ██████████ | ██████████ | ██████████ | ██████████ | ██████████ | ██████████ |
|                                                     AVG GPU UTL: 100.0% ██████████ |
+-----+

!CAUTION: SUPERUSER LOGGED-IN.

+-----+
| Processes:                                                                 root@97eadbf0622f |
| GPU      PID      USER   GPU-MEM %SM  %CPU  %MEM  TIME  COMMAND |
+-----+
|  0      2510 G      N/A    4.45MiB  0    N/A   N/A   N/A   No Such Process |
|  0      777956 C      N/A   38.52GiB 99    N/A   N/A   N/A   No Such Process |
|  0      777956 C      N/A   38.52GiB 99    N/A   N/A   N/A   No Such Process |
|  1      2510 G      N/A    4.45MiB  0    N/A   N/A   N/A   No Such Process |
|  1      777957 C      N/A   38.53GiB 99    N/A   N/A   N/A   No Such Process |
|  1      777957 C      N/A   38.52GiB 100   N/A   N/A   N/A   No Such Process |
|  2      2510 G      N/A    4.45MiB  0    N/A   N/A   N/A   No Such Process |
|  2      777958 C      N/A   36.81GiB 99    N/A   N/A   N/A   No Such Process |
|  2      777958 C      N/A   36.81GiB 100   N/A   N/A   N/A   No Such Process |
|  3      2510 G      N/A    4.45MiB  0    N/A   N/A   N/A   No Such Process |
|  3      777959 C      N/A   38.19GiB 100   N/A   N/A   N/A   No Such Process |
|  3      777959 C      N/A   35.57GiB 100   N/A   N/A   N/A   No Such Process |
|  2      777958 C      N/A   36.17GiB 75    N/A   N/A   N/A   No Such Process |
+-----+
|  3      2510 G      N/A    4.45MiB  0    N/A   N/A   N/A   No Such Process |
|  3      777959 C      N/A   34.75GiB 89    N/A   N/A   N/A   No Such Process |
|  2      777958 C      N/A   690.0MiB  0    N/A   N/A   N/A   No Such Process |
+-----+

```

Best experiment till now (0.700):

- "sfairXC/FsfairX-Gemma2-RM-v0.1"
- 125k lmsys pseudo + orpo dpo mix + stage 1 label smoothed data (totalling 271k)

Experiments selection final

- <https://www.kaggle.com/code/atharvaingle/best-sub-8bit?scriptVersionId=217784975> (CV: 0.710, LB: 700) [fix]
- <https://www.kaggle.com/code/atharvaingle/wsdm-cup-8bit-ensemble-stage-2-gemma-stage-1-llama?scriptVersionId=217795321> (CV: 0.709, LB: 700)
- <https://www.kaggle.com/code/atharvaingle/wsdm-cup-stage-2-submission?scriptVersionId=220444695> (CV: 0.710, LB: 0.699 (top))
- <https://www.kaggle.com/code/atharvaingle/wsdm-cup-stage-2-submission?scriptVersionId=220316926> (CV: 0.710, LB: 0.699)
- <https://www.kaggle.com/code/atharvaingle/best-sub-8bit?scriptVersionId=218537392> (CV: 0.715, LB: 0.697) (<https://wandb.ai/gladiator/WSDM-Cup-Multilingual-Chatbot-Arena/runs/4xjnr9ge>, stage 1 init stage 0 merged CV: 0.7051 wandb)
- <https://www.kaggle.com/code/atharvaingle/wsdm-cup-stage-2-submission?scriptVersionId=220398109> (kd, CV: 0.712, LB: 0.697) (v9 submitted with tta aug) (wandb CV: 0.7082)